

JWT Authentication in Node.js and Express

A Complete Practical Tutorial

1. Introduction to JWT Authentication

1.1 What JWT Is

JWT (JSON Web Token) is a secure way to authenticate users. When a user logs in, the server generates an encrypted token containing limited user information (e.g., user ID, username). This token:

- Is stored on the client-side (cookies or localStorage).
- Is sent with every protected API request.
- Is verified on every request to authorize the user.

1.2 JWT vs Session Authentication

Feature	Session	JWT
Storage	Server RAM / storage	Client-side storage (cookies/localStorage)
Load on server	Increases with users	Minimal
Persistence after server restart	Session lost	JWT remains on client
Storage capacity	Limited	Can store more data

2. Understanding JWT Token Structure

A JWT token has **three parts**, separated by dots (.):

```
header.payload.signature
```

2.1 Header

Contains algorithm information (default: HS256).

Example:

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

2.2 Payload

Contains actual JSON data (user ID, username, issued time).

Example:

```
{
  "sub": "123",
  "name": "Admin",
  "iat": 1516239022
}
```

2.3 Signature

Used to verify authenticity using the secret key.

3. Installing Required Packages

Install JSON Web Token and bcryptjs:

```
npm install jsonwebtoken bcryptjs
```

4. Creating User Model

Create file: **models/users.model.js**

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  username: { type: String, required: true, unique: true },
  email:    { type: String, required: true, unique: true },
  password: { type: String, required: true },
  created:  { type: Date, default: Date.now }
});

const User = mongoose.model("User", userSchema);
module.exports = User;
```

5. Creating Environment Secret

File: **.env**

```
JWT_SECRET=YahooBaba$123
```

Access inside Node.js:

```
process.env.JWT_SECRET
```

6. Creating User Routes (Register, Login, Logout)

File: **routes/users.routes.js**

```
const express = require('express');
const router = express.Router();
const User = require('../models/users.model');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');

require('dotenv').config();
```

6.1 User Registration Route

Steps:

1. Receive username, email, password.
2. Check if user already exists.
3. Hash password using bcrypt.
4. Save user in MongoDB.
5. Return success response.

```
router.post('/register', async (req, res) => {
  try {
    const { username, email, password } = req.body;

    const existingUser = await User.findOne({
      $or: [{ username }, { email }]
    });

    if (existingUser) {
      return res.status(400).json({ message: "Username or Email already exists" });
    }

    const hashPassword = await bcrypt.hash(password, 10);
```

```
    const newUser = new User({
      username,
      email,
      password: hashPassword
    });

    await newUser.save();
    return res.json(newUser);

  } catch (err) {
    return res.status(500).json({ message: err.message });
  }
});
```

6.2 User Login Route

Steps:

1. Validate username.
2. Compare password using `bcrypt.compare()`.
3. Generate JWT token using `jwt.sign()`.
4. Send token in response.

```
router.post('/login', async (req, res) => {
  try {
    const { username, password } = req.body;

    const user = await User.findOne({ username });

    if (!user)
      return res.status(404).json({ message: "User not found" });

    const isMatch = await bcrypt.compare(password, user.password);

    if (!isMatch)
      return res.status(400).json({ message: "Invalid credentials" });

    const token = jwt.sign(
      {
        userId: user._id,
        username: user.username
      },
      process.env.JWT_SECRET,
      { expiresIn: "1h" }
    );

    return res.json({ token });

  } catch (err) {
```

```
    return res.status(500).json({ message: err.message });
  }
});
```

6.3 Logout Route

JWT cannot be "deleted" server-side. Logout requires client-side removal.

```
router.post('/logout', (req, res) => {
  return res.json({ message: "Logout Successful" });
});
```

7. Creating JWT Middleware

File: **middleware/auth.js**

Middleware Responsibilities:

- Extract Bearer token from **Authorization** header.
- Remove "Bearer " prefix.
- Verify token using **jwt.verify()**.
- Pass control to next middleware or deny access.

```
const jwt = require('jsonwebtoken');
require('dotenv').config();

module.exports = async (req, res, next) => {
  try {
    const bearerHeader = req.headers['authorization'];

    if (!bearerHeader)
      return res.status(401).json({ message: "No token provided" });

    const token = bearerHeader.split(" ")[1];

    const user = jwt.verify(token, process.env.JWT_SECRET);

    req.tokenData = user;
    next();

  } catch (err) {
    return res.status(403).json({ message: "Invalid or expired token" });
  }
};
```

8. Protecting Student Routes Using Middleware

File: **index.js**

```
const auth = require('./middleware/auth');
const userRoutes = require('./routes/users.routes');
const studentRoutes = require('./routes/student.routes');

app.use('/api/users', userRoutes);
app.use('/api/students', auth, studentRoutes);
```

All student routes now require a valid token.

9. Testing in Postman

9.1 Register a User

POST

```
http://localhost:3000/api/users/register
```

Body (URL Encoded or JSON):

```
username: admin
email: admin@mail.com
password: 123456
```

9.2 Login

POST

```
http://localhost:3000/api/users/login
```

Returns:

```
{
  "token": "eyJh..."
}
```

9.3 Using Token to Access Protected Routes

GET

```
http://localhost:3000/api/students
```

Add Header:

```
Authorization: Bearer <your_token_here>
```

10. Twig Template Example (Replacement for EJS)

If your project uses a login UI, Twig example:

login.twig

```
<form action="/api/users/login" method="POST">
  <input type="text" name="username" placeholder="Enter username">
  <input type="password" name="password" placeholder="Enter password">
  <button type="submit">Login</button>
</form>
```

11. Summary

This tutorial covered:

- Understanding JWT, its structure, and purpose
 - Differences between JWT and Session authentication
 - Installing jsonwebtoken and bcryptjs
 - Creating User model and secure password hashing
 - Implementing Register, Login, and Logout APIs
 - Generating JWT tokens with `jwt.sign()`
 - Verifying JWT tokens with `jwt.verify()`
 - Building JWT-based middleware
 - Protecting Express routes
 - Testing authentication flow in Postman
 - Replacing EJS with Twig where applicable
-